

〈Blitz of Card〉

팀 : Last men standing

팀장 : 정현우(5820692)

팀원 : 김민준(5945928), 남현희(5945993), 정수홍(5645605)

소스 코드 : <https://github.com/LocomotiveMaker/Slay-The-Spire>

시연 동영상 : <https://youtu.be/fIS3S1p-QHA>

목차

1. 프로젝트 개요

- 작품 주제 선정 배경
- 벤치마킹 자료
- 개발 환경

2. 개발 프로그램

- 헤더 파일 다이어그램
- 다이어그램 설명
- UI

3. 개발 중 발생한 장애요인 및 문제 해결

- 장애요인 및 해결 과정

4. 결론

- 프로젝트 성과
- 실제 개발 일정 및 각 팀원 업무 분담
- 아쉬웠던 점

1. 프로젝트 개요

- 작품 주제 선정 배경

주제 : 덱 빌딩 전략을 이용한 전투 기반 로그라이크 게임

배경 : 기존 덱 빌딩 로그라이크 특유의 긴 플레이타임과

스트레스 요소가 큰 것을 발견하여

장르 본연의 재미는 유지하되,

실시간으로 진행되는 점을 차용하여 플레이 도중 속도감과 긴장감을 유지하기 위해서입니다.

- 벤치마킹 자료

Slay the Spire의 캐릭터 하나를 조작하며 카드를 사용하는 전투 구조 참고

Inscription의 전투 이후 카드 획득 및 강화 구조 참고

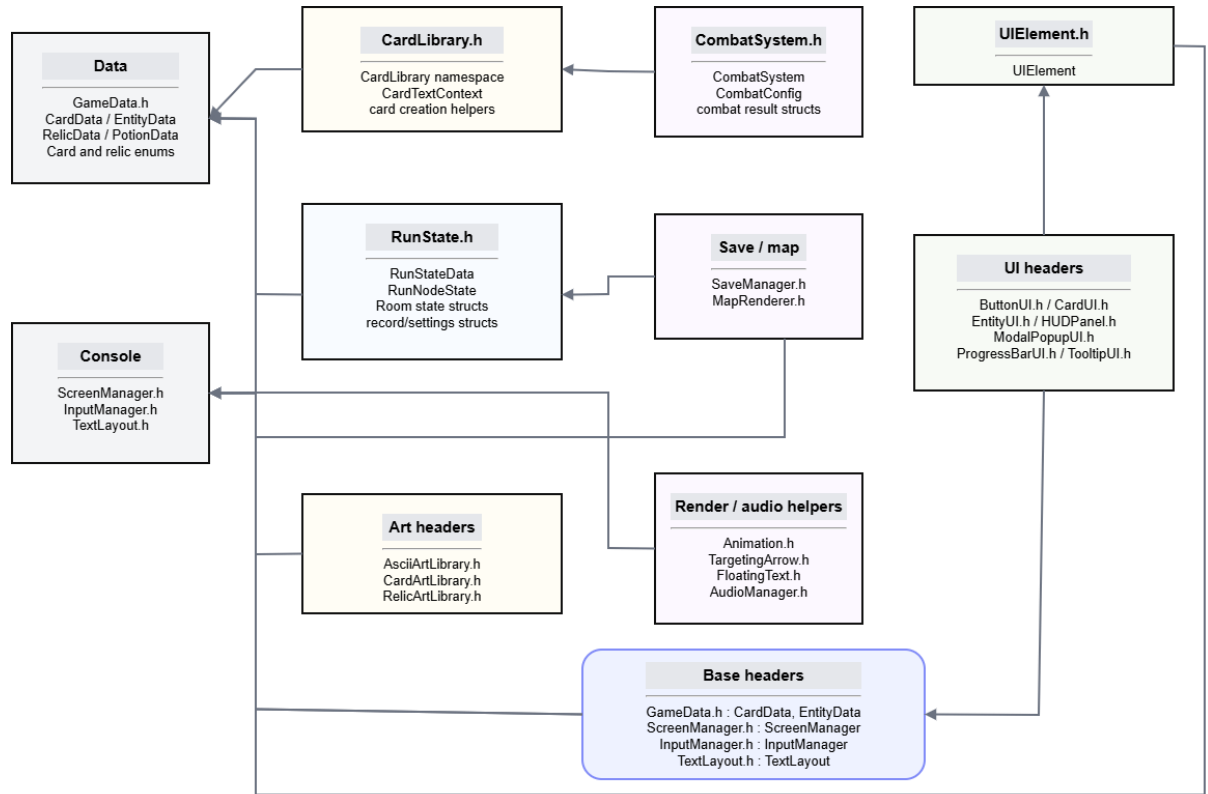
Clash Royale의 시간 비례 코스트 회복 방식 참고

- 개발 환경

- 개발 도구 : Visual Studio 2022, codex
- 플랫폼 : github, 챗gpt, gemini

2. 프로그램

- 다이어그램



각 상자는 헤더파일 및 클래스의 역할에 따라 유사한 파일들을 묶고, 이에 따른 상속 구조를 표시한 다이어그램입니다.

노란 상자는 출력 화면을 관리하는 파일들입니다.

파란 상자는 게임 진행을 관리하는 파일들입니다.

회색 상자는 체력, 공격력, 공격 시간 등 게임 내부 데이터를 관리하는 파일들입니다.

붉은 상자는 전투와 저장을 관리하는 파일들입니다.

- 다이어그램 설명

Data : GameData.h를 중심으로 게임 전체의 데이터를 관리하는 파일의 집합입니다. CardData, EntityData, RelicData, PositionData는 각각 카드, 적, 유물, 위치 정보를 관리하며, 이 곳에서 전투, UI, 게임 진행에 대한 데이터를 관리합니다.

주요 코드 :

```
struct CardData {           //카드 정보 기본형
    int id = 0;
    std::string name;
    int cost = 0;
    CardType type = CardType::Skill;
    CardTargetType targetType = CardTargetType::None;
};
struct EntityData { //적 정보 기본형
    std::string name;
    int currentHp = 0;
    int maxHp = 0;
    int block = 0;
    int strength = 0;
};
struct RelicData {         //유물 정보 기본형
    int id = 0;
    std::string name;
    std::string description;
};
```

Console : 콘솔 기반 입출력 기능을 중심으로 관리하는 파일의 집합입니다. ScreenManager.h는 화면 출력, InputManager.h는 마우스와 키보드 입력, TextLayout.h는 텍스트 정렬과 줄바꿈을 담당합니다.

주요 코드 :

```
// ScreenManager.h
class ScreenManager {    //화면 출력 관리
public:
    void Clear();
    void Render();
    void DrawString(int x, int y, const std::string& str, WORD color =
COLOR_WHITE);

// InputManager.h
class InputManager {    //마우스, 키보드 입력
public:
    void Update();
    bool IsLeftClickDown() const;
    int GetMouseX() const;
    int GetMouseY() const;
};
```

CardLibrary.h : 카드 생성과 카드 설명을 담당합니다.

주요 코드 :

```
namespace CardLibrary {
struct CardTextContext {
    int comboCount = 0;
    int playerBlock = 0;
    int currentEnergy = 0;
};
CardData CreateCardById(int id);
std::vector<CardData> BuildStarterPackCards(CardArchetype archetype);
}
```

RunState : 게임 한 판의 진행 상태를 관리합니다. 지도, 저장, 방 진행이 이 진행 상태를 중심으로 이루어집니다.

주요 코드 :

```
struct RunStateData {          //게임 내부 데이터
    int currentFloor = 0;      //층 수
    int gold = 99;            //기본 골드
    EntityData player = {};    //플레이어 캐릭터 정보
    std::vector<CardData> deck;    //덱 정보
    std::vector<RelicData> relics;
    std::vector<RunNodeState> nodes;

    BattleRoomState battleRoom = {};
    ShopRoomState shopRoom = {};
    EventRoomState eventRoom = {};
};

void CreateNewRun(RunStateData& run, std::uint32_t seed, int screenWidth, int screenHeight);
bool EnterNode(RunStateData& run, int nodeId);
```

CombatSystem.h : 손패, 덱, 에너지, 카드 사용, 적 행동, 승패 판정을 처리합니다.

주요 코드 :

```
// CombatSystem.h
class CombatSystem {
private:
    EntityData* player;        //플레이어 데이터
    EntityData* enemy;         //적 데이터
    std::vector<CardData> drawPile; //스프라이트 그리기
    std::vector<CardData> discardPile; //카드 불러오기
    std::vector<CardData> hand;    //패 구성

public:
    void StartBattle(const std::vector<CardData>& startingDeck);
    CombatFrameResult Update(float deltaTimeSec, float timeScale = 1.0f);
    CombatActionResult TryUseCard(int handIndex, CombatDropTarget target);
};
```

Art headers : 캐릭터/적/카드/유물의 아스키 아트를 관리합니다.

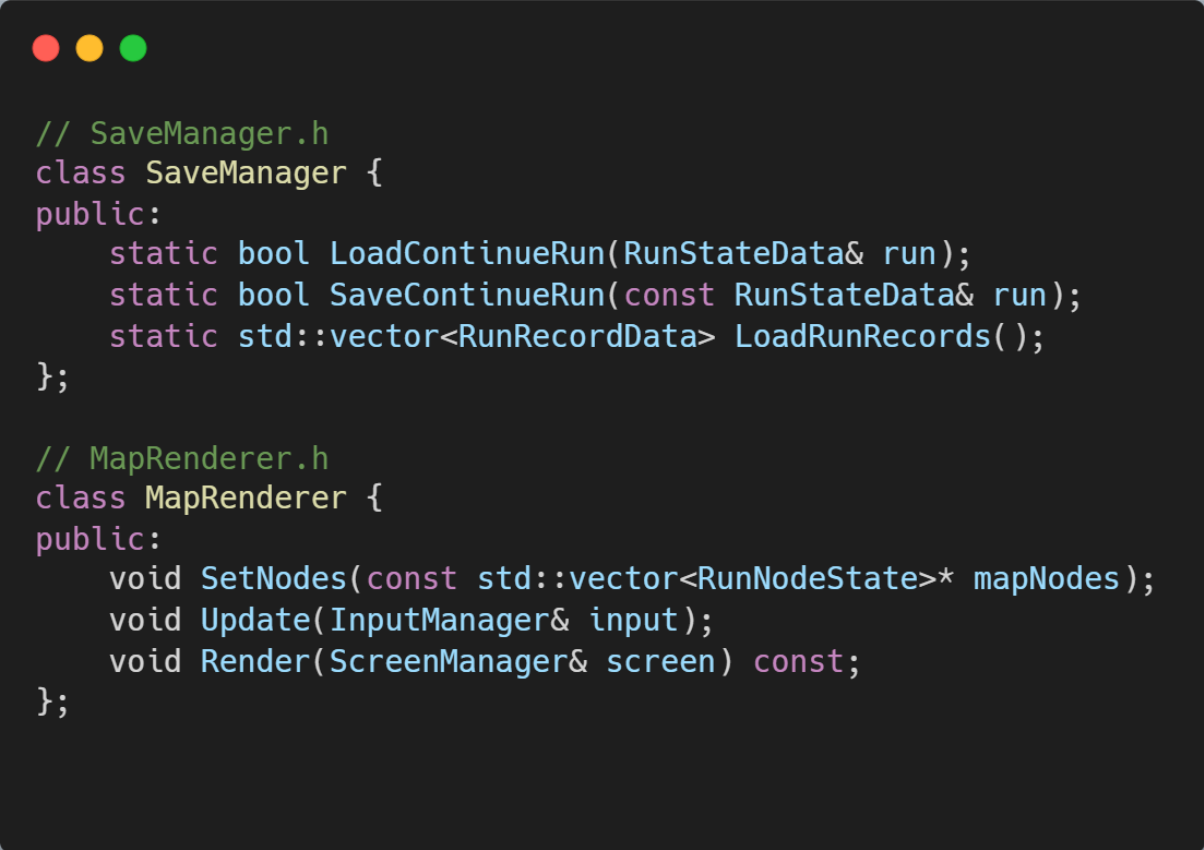
주요 코드 :

[illegible]

ID 단위로 저장된 아스키 아트를 불러옵니다. 위 코드에 나타난 아스키 코드는 예시로, 다른 아스키 아트도 이 파일에 저장되어 있습니다.

Save/map : 게임 진행도 저장과 지도의 노드 구조를 관리합니다.

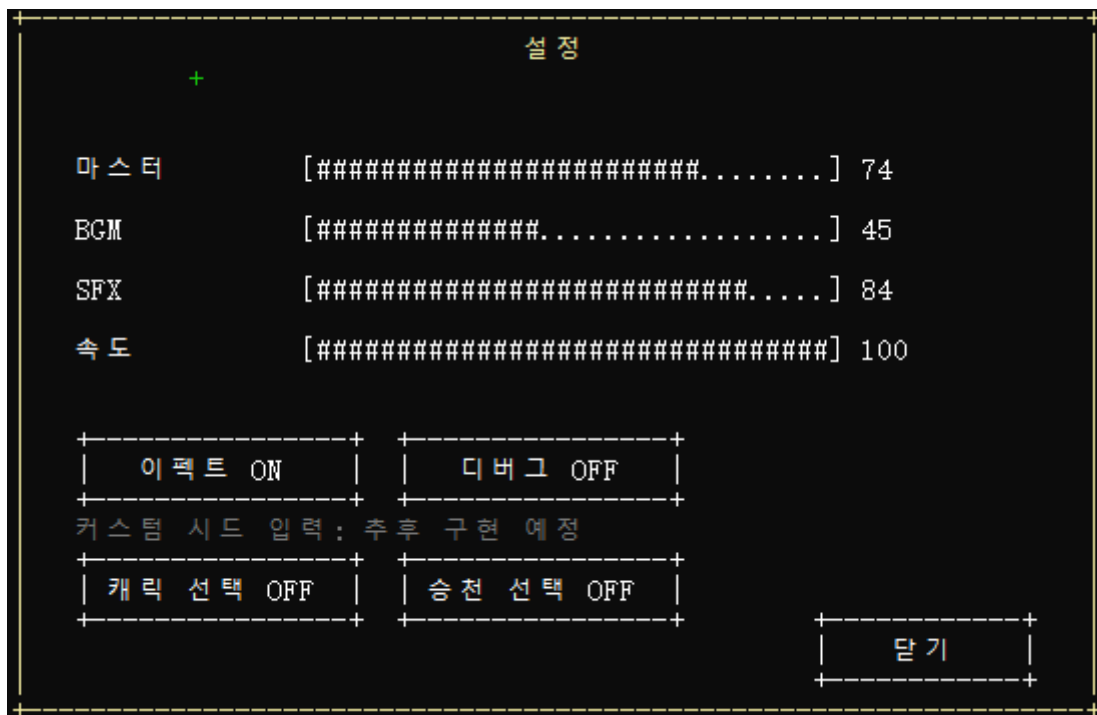
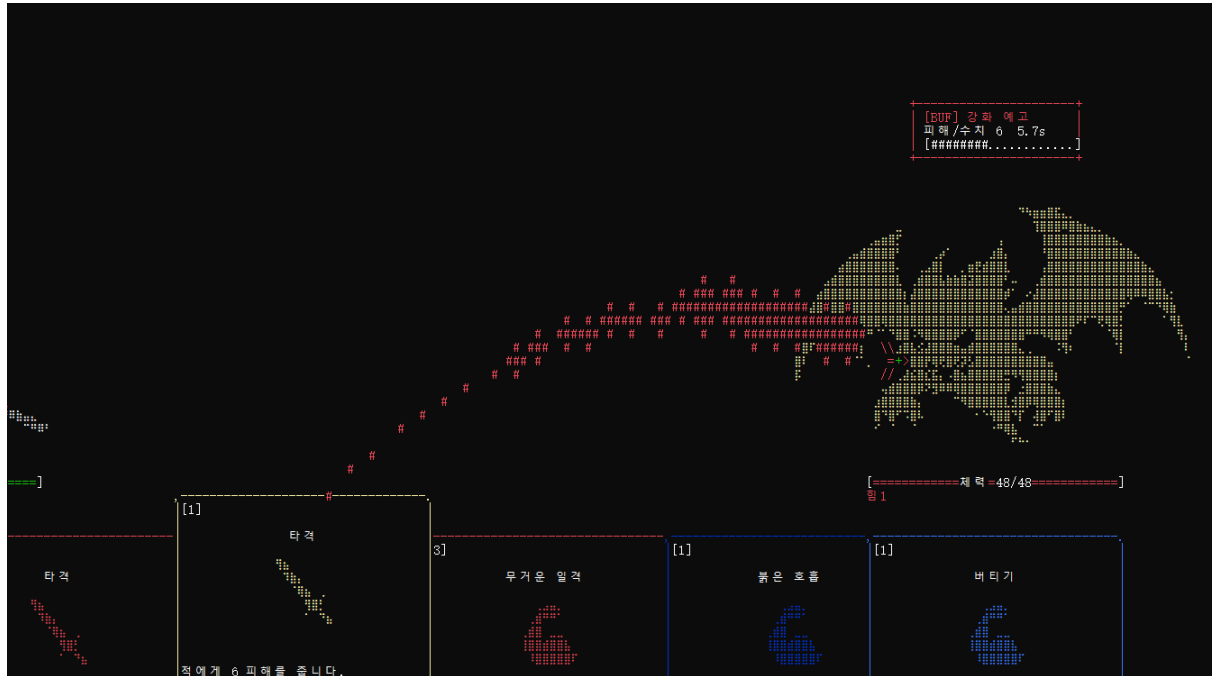
주요 코드 :



```
// SaveManager.h
class SaveManager {
public:
    static bool LoadContinueRun(RunStateData& run);
    static bool SaveContinueRun(const RunStateData& run);
    static std::vector<RunRecordData> LoadRunRecords();
};

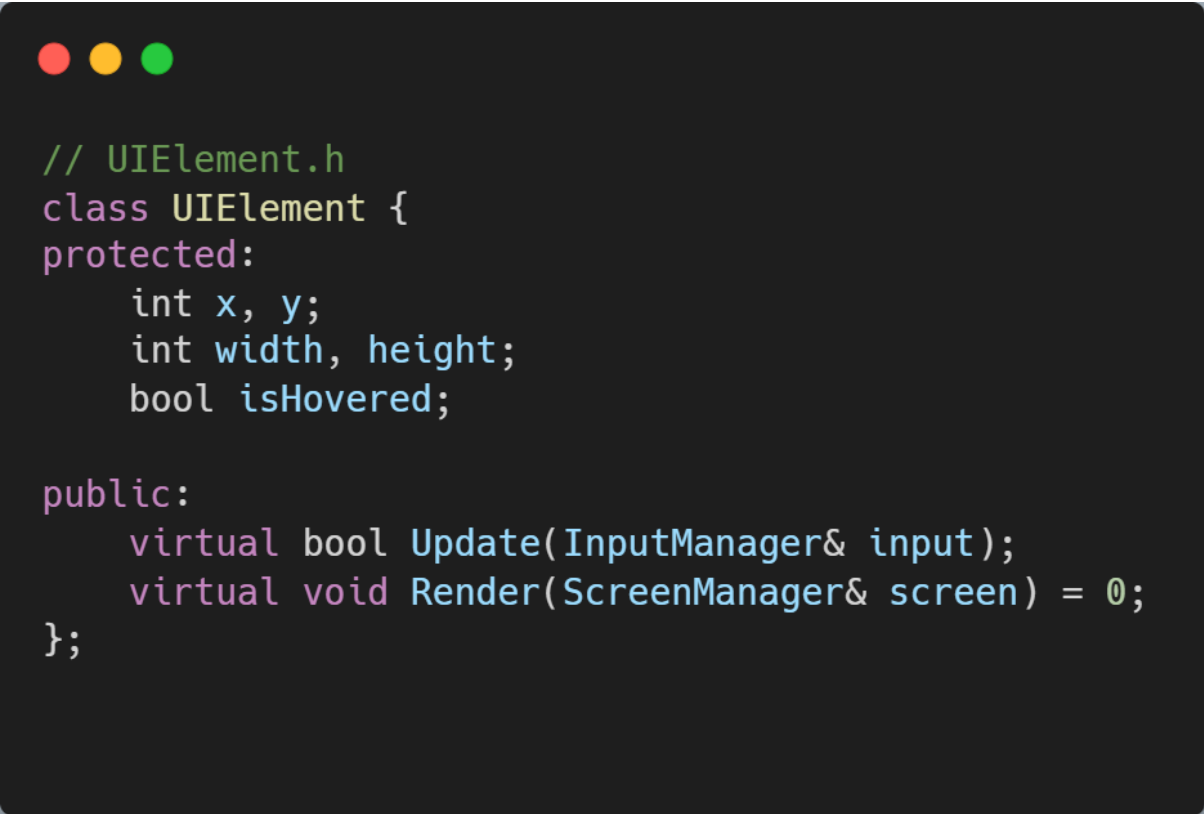
// MapRenderer.h
class MapRenderer {
public:
    void SetNodes(const std::vector<RunNodeState>* mapNodes);
    void Update(InputManager& input);
    void Render(ScreenManager& screen) const;
};
```

Render/audio helpers : 전투 효과, 타겟팅 화살표, 배경음 및 효과음을 관리합니다.



UIElement.h : 모든 UI클래스의 기본형인 위치, 크기, 업데이트 기능을 제공합니다.

주요 코드 :



```
// UIElement.h
class UIElement {
protected:
    int x, y;
    int width, height;
    bool isHovered;

public:
    virtual bool Update(InputManager& input);
    virtual void Render(ScreenManager& screen) = 0;
};
```

UI headers : 실제 화면 구성 요소의 집합입니다. UIElement.h에 기능을 상속받아 각 클래스별 UI를 화면에 표시합니다.

주요 코드 :

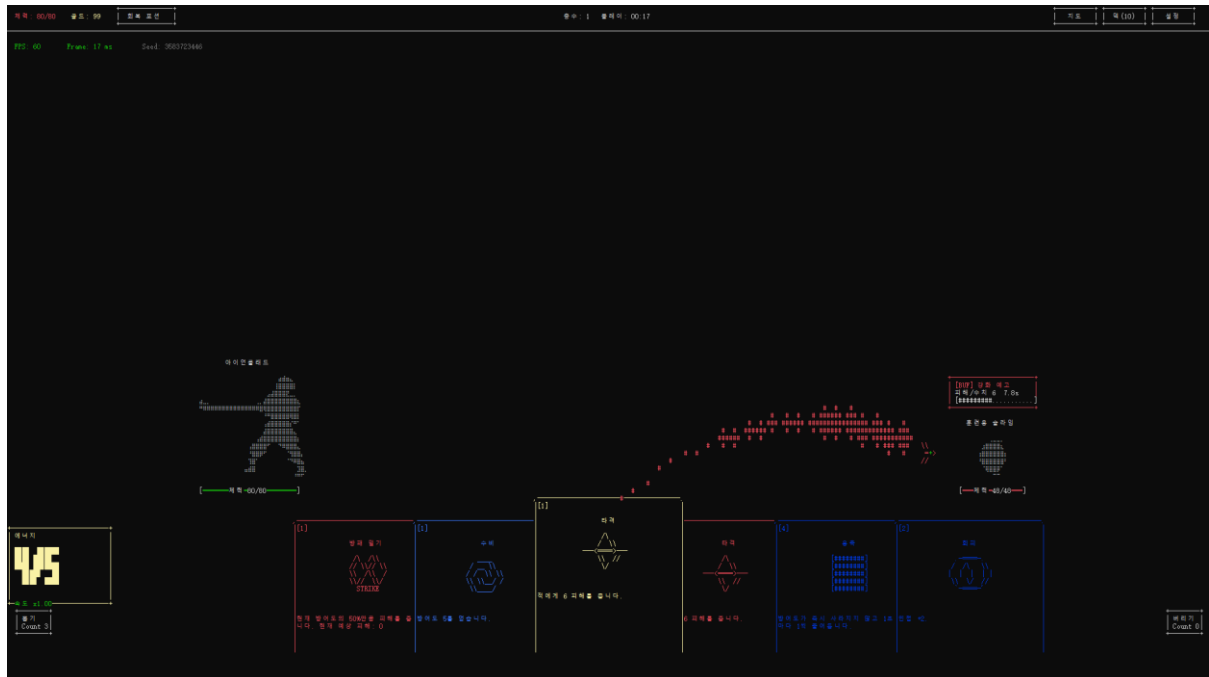
```
// ButtonUI.h
class ButtonUI : public UIElement {    //UIElement를 상속
public:
    bool Update(InputManager& input) override; //Update함수 오버라이드
    void Render(ScreenManager& screen) override; //Render함수 오버라이드
};

// CardUI.h
class CardUI : public UIElement {    //UIElement를 상속
private:
    CardData* data;
public:
    void Render(ScreenManager& screen) override;
};

// EntityUI.h
class EntityUI : public UIElement {    //UIElement를 상속
private:
    EntityData* data;
    ProgressBarUI* healthBar;
};
```

2. 프로그램

- UI



플레이어가 주요하게 봐야 할 카드나 코스트 같은 주요 정보를 하단에 배치하고, 가운데에는 현 상황을 알 수 있도록 플레이어 캐릭터와 적 캐릭터를 배치하였습니다.

상단에는 전투와 직접적인 영향이 적은 아이템과 설정을 배치하였습니다

3. 개발 중 발생한 장애요인 및 문제 해결

- 장애요인 및 해결 과정

1. 게임의 플레이 화면이 다른 팀원의 터미널 환경에서 깨어진 상태로 보이는 문제 발생

> 설정의 기본 터미널 응용 프로그램을 Windows 터미널로 교체함으로 해결하였습니다



2. 아스키 아트 스프라이트를 코드에 적용 시 출력 화면에서 올바르게 출력되지 않는 문제 발생

> 각 아스키 아트를 코드의 출력 환경에 맞게 변경하여 해결하였습니다

3. 특정 카드로 다른 카드를 지정했을 때 게임이 멈추는 현상 발생

> 수비의 대상 지정 코드에서 오류가 발생한 것으로 코드를 수정하여 해결하였습니다.

4. 결론

- 프로젝트 성과

카드 덱 빌딩 프로그램을 제작하며 카드 게임에서 흥미를 끌 만한 게임이 되기 위해서는 게임의 전체 규칙은 단순하고 잘 알 수 있게 만들며, 카드의 기믹을 다양하게 만들어 플레이어의 선택지를 늘리되 게임의 진행에 따라 자연스럽게 선택지가 많아지는 방식이 플레이어의 재미를 끌어낼 수 있는 방법이라는 것을 알게 되었습니다.

게임 플레이 중 빈번하게 변경되고 교체되는 덱을 저장하는 코드를 만들며 유동적으로 변화하는 데이터의 저장에 대한 구조를 잘 이해할 수 있었습니다.

카드의 테마를 반영한 스프라이트가 추가적인 설명을 요구하지 않아도 유저가 이해하기에 도움을 줘, 덱을 완성하기 무리가 없었습니다. 따라서 카드의 스프라이트 또한 유저의 이해를 돕기 위해 필수적이라는 것을 알았습니다.

4. 결론

- 아쉬웠던 점

개발을 시작할 때 글자 크기를 작게 잡아 카드의 설명을 읽기에 어려움이 있었습니다. 이를 해결하기 위해서는 게임 전체의 스프라이트 구조를 교체해야 하여 해결할 수 없었습니다.

스프라이트 중 애니메이션으로 구현된 오브젝트가 유저가 보기에는 한눈에 알아볼 수 없었다는 것이 아쉬웠습니다.

게임의 음악과 효과음 부분이 대다수가 슬더스의 음악을 차용하여 프로젝트만의 음악을 만들지 않았던 점입니다.

게임의 밸런스 조정이나 적 패턴 조정 등 근본적인 재미를 끌어올리지 못했다는 점이 아쉽습니다.

전시를 고려하여 제작하였기에 게임의 후반부 콘텐츠의 준비가 부실하였습니다. 개발을 이어간다면 후반부 콘텐츠에 더 집중하여 만들고 싶었습니다.

4. 결론

- 실제 개발 일정 및 팀원 주요 업무 분담

4/16 ~ 5/1 : 핵심 프로그램 제작

5/1 ~ 5/8 : 핵심 스프라이트 제작 및 적용

5/9 ~ 5/15 : QA진행 및 카드 내부 스프라이트 제작

5/16 ~ 5/24 : 스프라이트 적용 및 프로젝트 완성

정현우 : 프로젝트 회의 진행 및 팀원 업무 분배

남현희 : 주요 프로그램 제작

정수홍 : 카드 및 캐릭터 스프라이트 제작

김민준 : 몬스터 및 유물 스프라이트 제작